

Final Project Report

Aaron Bussche

David Magaram

Jason Kasdiran

Group 28

May 25, 2026

Abstract

Final Report: Aim for a max 10 pages document, excluding references and contributions of team members. We will not subtract points for small deviations from the 10 pages, but keep in mind that if your report is much longer, there is likely an issue with conciseness / data presentation somewhere!

Introduction

Neural Cellular Automata (NCAs) are small, shared-weight networks that grow images from a single seed pixel and can regenerate after damage, making them a promising model for robust, self-organizing systems. Their perceived robustness has led to NCAs being deployed as a defensive component in larger architectures: Xu et al. [2] use NCA layers as plug-and-play adaptors inside Vision Transformers to make them more resistant against adversarial samples. Whether NCAs themselves can be subverted is therefore not just of theoretical interest but relevant to the security of systems that rely on them.

Randazzo et al. [3] and Cavuoti et al. [4] demonstrated targeted attacks that, for example, can make a generated lizard image lose its tail or change color. However, every published attack on NCAs relies on white-box, gradient-based methods that require full access to the model’s weights and backpropagation through the rollout. Robustness claims are so far tested only against this strong threat model. Additionally, focusing on the nature-inspired roots of NCAs, a white-box is not what a realistic adversary, for instance in a biological or bioengineering setting, would plausibly have access to.

We therefore ask: *can a query-only, black-box attacker using evolutionary search reproduce the same targeted reprogramming of a Growing NCA, and how close does it get to the gradient-based ceiling?* We attack the NCA via a 16×16 state-perturbation matrix applied to every cell’s 16-

dimensional hidden state at every timestep, the same attack surface used by Randazzo et al. Our attacker observes only the final RGB output and optimizes the L2 distance to a chosen target image using CMA-ES, without ever accessing model weights, hidden channels, or gradient information.

If Growing NCAs can be influenced by this attack in a similar way as through white-box approaches, they may be less robust than priorly assumed. Otherwise, they may be safe from more realistic attack types and to be used in public-facing, but not open-weights, ML systems.

Related Work

Mordvintsev et al. [1] introduced the Growing NCA model and showed that, through a sample pool training strategy and exposure to damage during training, the learned local update rules become robust enough to regenerate damaged parts. Due to this robustness, NCAs have also been used defensively: Xu et al. [2] inserted NCA layers between vision transformer blocks to serve as an armor against adversarial attacks.

On the offensive side, Randazzo et al. [3] investigated white-box vulnerabilities of Growing NCAs and showed that, while they are more resistant to local injections, they can still be reprogrammed using global state perturbations applied to every cell at every step. Similarly, Cavuoti et al. [4] demonstrated that injecting a small number of adversarial cells into the grid can drastically alter the grown image. Both studies rely on gradient-based optimization with full access to model weights.

Covariance Matrix Adaptation Evolution Strategy (CMA-ES), introduced by Hansen [5], is an algorithm for non-linear, non-convex black-box optimization that maintains a population of candidates and iteratively shifts them toward high-fitness regions. While evolutionary black-box attacks are commonly used to evaluate traditional image classifiers, they have not yet been applied as an adversarial tool against cellular automata.

Methodology and Approach

Motivation

To assess the true vulnerability of NCAs under a realistic threat model, we constrain the attacker to a strict black-box setting. The attacker cannot access the internal weights of the pretrained NCA, nor can they perform automatic differentiation through the grid’s temporal evolution. Instead, the attacker can only provide an initial state (or intervening perturbation) and observe the final converged visual output. Because the fitness landscape of an NCA rolled out over hundreds of steps is highly non-convex and non-differentiable from a black-box perspective, we utilize CMA-ES. This evolutionary strategy is well-suited for our setup because it is derivative-free, robust to noisy objective functions, and scales well to our specific parameter space.

Experimental Setup

We target a standard pretrained Growing NCA model optimized to generate a specific 2D image from a single seed pixel (e.g., the "lizard" emoji model). The NCA operates on a grid of cells, each possessing a continuous state vector $x \in \mathbb{R}^{16}$.

Following the framework established by Randazzo et al. [3], we restrict our attack to a global, linear state perturbation. At each timestep of the simulation, before the NCA’s convolutional update rules are applied, every cell’s state vector is multiplied by a global transformation matrix $M \in \mathbb{R}^{16 \times 16}$. To maintain stability in the cellular dynamics, we restrict M to be a symmetric matrix.

A symmetric 16×16 matrix contains exactly 136 unique parameters (the upper triangular elements, including the diagonal). This 136-dimensional vector space forms the search space for our CMA-ES algorithm. To evaluate the impact of the starting distribution on the evolutionary search, we initialize the population center from three distinct points:

- **The Identity Matrix:** This represents a "zero-damage" starting point, allowing the evolutionary search to incrementally discover minimal perturbations.
- **A Random Matrix:** Initialized via a standard normal distribution to test the algorithm’s ability to converge from an arbitrary, highly destructive state.
- **The Baseline Gradient Matrix:** The final output matrix derived from the white-

box gradient descent method used by Randazzo et al., establishing a benchmark for convergence speed and local minimum trapping.

For the initial standard deviation (σ_0), which dictates the initial search volume, we conduct a sweep across multiple magnitudes (e.g., $10^{-2}, 10^{-3}, 10^{-4}$). A smaller σ_0 is critical when starting from the identity matrix to prevent immediate catastrophic failure of the NCA rules.

Population size.

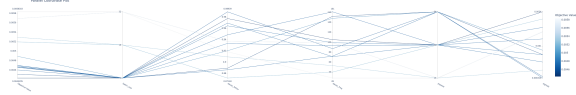


Figure 1: The first optuna sweep, 30 times 20 minutes with early pruning after 5 minutes



Figure 2



Figure 3: Second optuna sweep with more limited options, winner from the first round re-introduced, 20 times 20 minutes with early pruning after 5 minutes.



Figure 4

Our final algorithm uses the following parameters: [Insert final chosen parameters here after running your grid search/ablations, e.g., $\lambda = 64, \sigma_0 = 0.0002$].

sigma0	0.0048
batch_size	4
popsize	64
decay_factor	0.9
decay_freq	175

Evaluation Metrics

Evaluating the success of an adversarial attack on a dynamic system like a Growing NCA requires moving beyond simple objective function

minimization. To comprehensively assess the efficacy, reliability, and visual quality of the CMA-ES black-box attack against the white-box gradient baseline, we employ a multi-faceted evaluation strategy encompassing statistical, perceptual, temporal, and frequency-domain analyses.

Results

Results. Ensure to perform a correct experimental analysis (e.g. repeated experiments, statistical analysis of results). Verbally describe your results in this section and make sure to reference the corresponding figures/tables.

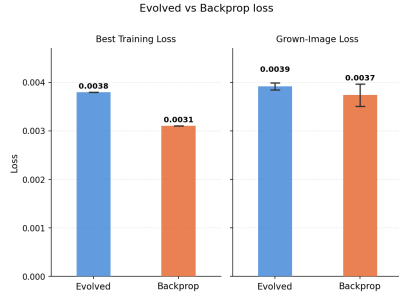


Figure 5: The best loss achieved during training and the averaged loss of the final model after training both of our implementation and the gradient-based implementation after 30 minutes of training time.

While the gradient-based model reaches better results in the same time, the loss of the NCA-ES trained adversarial attack is not too far removed. It is interesting to note that the gradient-based attack has a higher standard deviation than the evolved one.

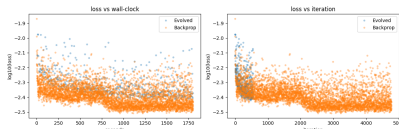


Figure 6

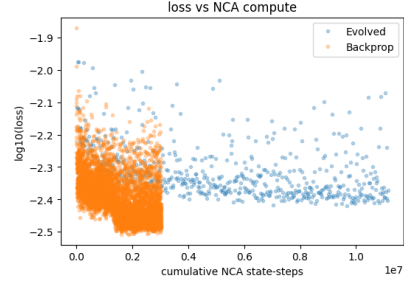


Figure 7

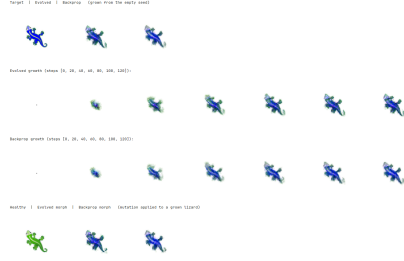


Figure 8

Discussion

Discussion. A thorough discussion based on the analysis of the results, including limitations and future directions (What additional enhancements could be made? What future work should be done?)

Conclusion

Conclusions. Is your line of work worth pursuing?

Appendix

References

- [1] A. Mordvintsev, E. Randazzo, E. Niklasson, and M. Levin, “Growing neural cellular automata,” *Distill*, vol. 5, no. 2, e23, 2020. [Online]. Available: <https://distill.pub/2020/growing-ca> doi: 10.23915/distill.00023.
- [2] Y. Xu, T. Zhang, and S. Süssstrunk, “AdaNCA: Neural cellular automata as adaptors for more robust vision transformer,” *Advances in Neural Information Processing Systems*, vol. 37, pp. 25709–25746, 2024.
- [3] E. Randazzo, A. Mordvintsev, E. Niklasson, and M. Levin, “Adversarial reprogramming of neural cellular automata,” *Distill*, vol. 6, no. 5, e00027-004, 2021. [Online]. Available: <https://distill.pub/selforg/2021/adversarial> doi: 10.23915/distill.00027.004.
- [4] L. Cavuoti, F. Sacco, E. Randazzo, and M. Levin, “Adversarial takeover of neural cellular automata,” in *Artificial Life Conference Proceedings 34*, 2022, p. 38. MIT Press.

- [5] N. Hansen and A. Ostermeier, “Completely derandomized self-adaptation in evolution strategies,” *Evolutionary Computation*, vol. 9, no. 2, pp. 159–195, 2001.

- [6] S. Greydanus, “Studying growth with neural cellular automata,” blog post and PyTorch implementation, 2022. [Online]. Available: <https://greydanus.github.io/2022/05/24/studying-growth>

Author Contribution

Author contribution: describe each team member contributions to the project.

Code

Include a link to a github repository with implementation of algorithms developed and used during the project. The repository should include source code, all data necessary to run the program (think instructions for installing external software, package versions/virtual environment, etc); a short manual describing how to use/run the program; a sample run, screenshot, or other indication of system behavior.